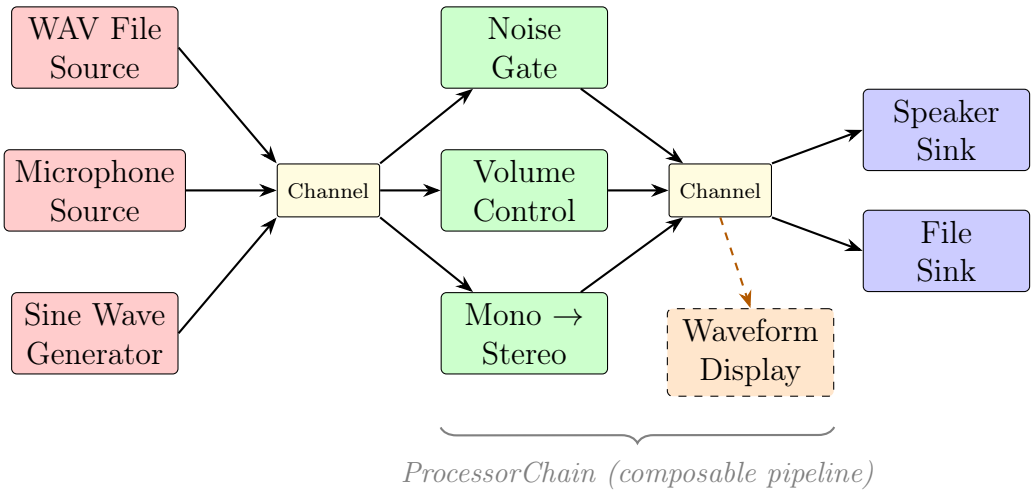
Programming Paradigms (Neapolis University Pafos)
Assignment 3: Audio Processing System

April 2026



1 Introduction

In this assignment you will design and implement a **versatile audio processing system** in Java that demonstrates core OOP principles in a realistic, non-trivial domain.

The system processes audio as a stream of **Pulse-Code Modulation (PCM) samples** flowing through a configurable pipeline: *sources* produce audio data, *processors* transform it, and *sinks* consume the result. All components communicate via thread-safe *channels*, and the pipeline is assembled using the *Builder* pattern.

Your system must handle audio at the **sample level** — understanding that a "byte array" is really a sequence of 16-bit signed integers in little-endian byte order.

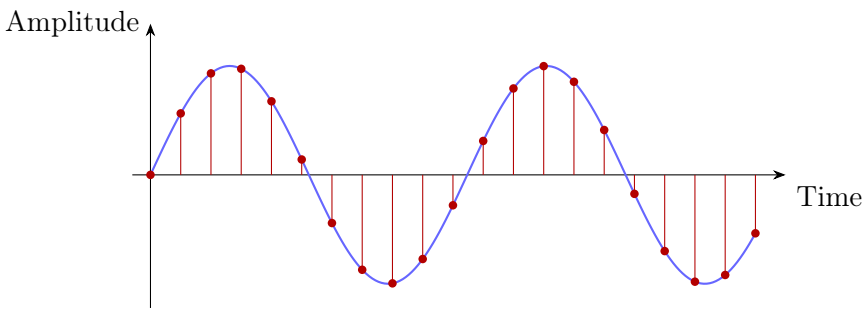
Use the provided Java-project template as your starting point. It contains interfaces, core infrastructure, and one working example (VolumeControlStrategy with proper PCM handling) to get you started.

2 Background: PCM Audio Fundamentals

Before writing audio processors, you must understand how digital audio is represented in memory.

2.1 What is PCM?

Pulse Code Modulation (PCM) is the standard way to represent audio digitally. An analog sound wave is sampled at regular intervals and each sample is quantized to a fixed number of bits.



Each red dot is one **sample** (e.g., a 16-bit signed integer)

2.2 Key Parameters

- **Sample Rate**: Number of samples per second (e.g., 44 100 Hz for CD quality).
- **Bit Depth**: Bits per sample. **16-bit signed** is standard. Values range from $-32\,768$ to $+32\,767$.
- **Channels**: Mono (1 channel) or Stereo (2 channels, interleaved: L, R, L, R, ...).
- **Byte Order**: 16-bit samples are stored as 2 bytes in **little-endian** order (low byte first).

2.3 Reading a 16-bit Sample from a Byte Array

```
// bytes[i] is the low byte, bytes[i+1] is the high byte
int sample = (bytes[i] & 0xFF) | (bytes[i + 1] << 8);

// Writing back after processing:
bytes[i]     = (byte) (sample & 0xFF);
bytes[i + 1] = (byte) ((sample >> 8) & 0xFF);
```

This is the fundamental operation you will use in every audio processor. **Ignoring byte order or treating individual bytes as samples is a common mistake** that produces noise instead of processed audio.

3 OOP Principles and Design Patterns

This project exercises multiple OOP concepts and design patterns:

1. **Encapsulation**: Audio format details, buffer management, and thread synchronization are hidden inside Channel and AudioStream implementations.
2. **Inheritance & Interfaces**: `AudioStream`, `AudioProcessingStrategy`, and `DataPin` define contracts; concrete classes provide implementations.
3. **Polymorphism**: Different sources, processors, and sinks are interchangeable through their interfaces.
4. **Strategy Pattern**: Audio processors implement `AudioProcessingStrategy`, allowing algorithms to be swapped at runtime.
5. **Observer Pattern**: `ObservableChannel` notifies `DataPin` subscribers when data arrives, enabling reactive data flow.

6. **Composite Pattern**: `ProcessorChain` composes multiple strategies into one, applying them sequentially.

7. **Builder Pattern**: `AudioPipeline.Builder` provides a fluent API for assembling pipelines declaratively.

8. **Factory Pattern**: Create sources, sinks, and processors through factory methods for cleaner instantiation.

4 Core Components

4.1 Audio Stream Interface

```java
public interface AudioStream {
    byte[] read() throws IOException;
    void write(byte[] data) throws IOException;
    void close() throws IOException;
}
```

Foundation for any class that handles audio I/O. Implementing classes provide the specifics: `FileAudioStream` for files, `MicrophoneAudioStream` for microphone input, `NetworkAudioStream` for network streaming, etc.

4.2 Channel System

Channel buffers audio data between components using a `BlockingQueue`, enabling asynchronous, thread-safe producer-consumer flow. Subchannels carry metadata (e.g., timestamps, checksums, audio format info).

ObservableChannel extends Channel with the Observer pattern: when data arrives, all subscribed `DataPin` instances are notified.

4.3 Audio Processing Framework

The **Strategy Pattern** powers the processing framework:

```java
public interface AudioProcessingStrategy {
    byte[] processAudio(byte[] inputData);
}
```

Each concrete strategy implements a specific audio transformation. The provided `VolumeControlStrategy` demonstrates proper 16-bit PCM sample handling with overflow clipping — use it as a reference for your own processors.

ProcessorChain implements the **Composite Pattern**, chaining multiple strategies:

```java
ProcessorChain chain = new ProcessorChain()
    .add(new NoiseGateStrategy(500))
    .add(new VolumeControlStrategy(1.5))
    .add(new MonoToStereoStrategy());

byte[] result = chain.processAudio(inputData);
```

4.4 Pipeline Composition (Builder Pattern)

The `AudioPipeline` class provides a fluent **Builder API** for assembling complete audio pipelines:

```
Pipeline pipeline = AudioPipeline.builder()
    .source(new WAVFileReader(file, inputChannel))
    .channel(inputChannel)
    .processor(processorChain)
    .channel(outputChannel)
    .sink(new SpeakerConsumer(outputChannel))
    .sink(new FileWriterConsumer(outFile, outputChannel))
    .build();

pipeline.start();
// ... audio flows through the pipeline ...
pipeline.stop();
```

Students must complete the `AudioPipeline` implementation, handling thread management, graceful shutdown, and error propagation.

4.5 Pins and Observer Pattern

`DataPin` is the observer interface; `AudioProcessorPin` is a concrete observer that retrieves data from an `ObservableChannel`, applies a processing strategy, and forwards the result. This enables reactive, event-driven data flow through the pipeline.

4.6 Sine Wave Generator

The blueprint includes a `SineWaveGenerator` — an audio source that produces a mathematically perfect sine wave at a configurable frequency. Use it to **test your processors without needing actual WAV files**:

```
// Generate 2 seconds of 440 Hz tone (A4 note)
SineWaveGenerator gen = new SineWaveGenerator(440.0, 2.0);
byte[] audioData = gen.generate();
```

4.7 Utility Components

- **WAVUtils**: Read and write WAV file headers (44-byte RIFF format). Provided in the blueprint — essential for any file I/O.
- **SystemLogger**: Centralized logging utility.
- **Converter⟨T, R⟩**: Generic conversion interface for transformations like Speech-to-Text.

5 The Problems

Note: we reserve the right to deduct points for incomprehensible code, careless project structure, or solutions that clearly do not understand the underlying audio representation (e.g., treating individual bytes as samples).

5.1 Problem 1: Core Audio Pipeline (20 points)

Implement a working end-to-end audio pipeline.

1. Implement at least **2 audio sources**. Choose from:
   - WAV file reader (reading from `.wav` files on disk)
   - Microphone capture (real-time input from system microphone)
   - Network stream (receiving audio over TCP/UDP)
   - Sine wave / tone generator (already provided as reference)
2. Implement at least **2 audio consumers**. Choose from:
   - WAV file writer (writing processed audio to `.wav` files)
   - Speaker output (real-time playback through system speakers)
   - Network sender (streaming audio over the network)
3. Create a **complete working example** that reads audio from a source, processes it through at least one processor, and sends the result to a consumer.
4. All sources and consumers must use the Channel infrastructure for data flow and handle **audio format metadata** properly (sample rate, bit depth, channels).

5.2 Problem 2: Audio Processors and Pipeline Composition (25 points)

Implement audio processing strategies and composable pipelines.

1. Implement a few from **4 audio processing strategies**, all working with proper 16-bit signed PCM samples:
   - **Volume Control** — scale sample amplitude with overflow clipping (provided as reference in the blueprint)
   - **Noise Gate** — zero out samples below an amplitude threshold
   - **Mono ↔ Stereo Conversion** — duplicate mono samples to both channels, or average stereo channels to mono
   - **One non-trivial DSP algorithm** — choose one:
     - *Echo / Delay*: mix signal with a delayed copy of itself
     - *Reverb*: simulate room acoustics (e.g., Schroeder or Freeverb algorithm)
     - *Speed / Pitch modification*: resample audio with interpolation
     - *Simple Equalizer*: boost/cut frequency bands using filters
     - *Compressor / Limiter*: reduce dynamic range
     - *Fade In / Fade Out*: gradually change volume over time
     - Or propose your own variant!
2. Complete the `ProcessorChain` implementation to chain multiple strategies via the Composite Pattern. Verify that the order of processors matters (e.g., noise gate before volume boost gives different results than the reverse).
3. Complete the `AudioPipeline.Builder` to declaratively construct pipelines with sources, processor chains, and multiple sinks.

5.3 Problem 3: Testing (25 points)

Write comprehensive tests for each component.

1. **Unit tests** for:
   - Each audio processing strategy (verify correct sample transformation, clipping behavior, edge cases with silence / maximum amplitude)
   - Channel and ObservableChannel (buffering, blocking, subchannel operations, observer notifications)
   - ProcessorChain (verify order of application, empty chain, single processor)
   - Sources and consumers (mock hardware where necessary)
2. **Integration tests** for:
   - Complete pipeline: source → processor chain → consumer
   - Use `SineWaveGenerator` as a deterministic source for reproducible tests
   - Verify that audio data passes through the pipeline correctly
3. Use **JUnit 5** and **Mockito** (both included in the blueprint POM). The blueprint includes sample tests — follow their patterns.

5.4 Problem 4: UI with Real-time Visualization (30 points)

Build a graphical interface for configuring and monitoring the audio pipeline.

1. **Pipeline Configuration UI** (15 points):
   - Select audio source type and configure its parameters (e.g., file path, microphone device, network address)
   - Select and configure processors with adjustable parameters (e.g., volume slider, noise threshold, echo delay time)
   - Select audio consumer(s) and configure output
   - Start / Stop controls for pipeline execution
   - Status display and logging panel
2. **Real-time Audio Visualization** (15 points):
   - Display a **waveform view** (amplitude vs. time) that updates in real-time as audio flows through the pipeline
   - *OR* display a **spectrum / level meter** (frequency or amplitude bars)
   - The visualization must tap into the channel data without disrupting the audio flow (read from a subchannel or use the Observer pattern)
   - Update smoothly at a reasonable frame rate ($\geq$10 FPS)
3. Use **JavaFX** (included in the blueprint) or another Java UI library of your choice (Swing, LibGDX, etc.). If you choose an alternative, justify it briefly in your README.

Tip: JavaFX Canvas or a custom Node works well for waveform drawing. Use `Platform.runLater()` for thread-safe UI updates from audio processing threads.

5.5 Bonus Challenge (not graded)

The most educational challenge is not something we can auto-grade or peer assess, so we encourage doing it even though it will not count toward your grade:

- Reimplement (part of) your solution in a different OOP language: Kotlin, Scala, Rust, C#, or Python.
- Implement pipeline serialization: save and load pipeline configurations to/from JSON or XML files.
- Add a visual pipeline editor where users drag-and-drop source/processor/sink nodes and connect them with wires (one of the best student solutions from last year did this!).
- Integrate a real speech recognition library (e.g., Vosk) as a consumer.

6 Choosing a Framework

This project uses the **Spring Framework** for dependency injection and configuration management. Spring Boot provides auto-configuration, bean management, and externalized properties.

6.1 Alternatives

If you prefer a different framework, you may use one of these alternatives (document your choice in README):

1. **Micronaut** — lightweight, fast startup, minimal memory footprint.
2. **Quarkus** — Kubernetes-native, optimized for containers.
3. **Helidon** — Oracle's microservices library with reactive support.
4. **Vert.x** — event-driven, non-blocking toolkit.
5. **Plain Java** — no framework, manual dependency wiring. Acceptable if you demonstrate clean architecture.

7 Grading Criteria

Problem	Points	Key Criteria
1. Core Pipeline	20	Working sources, consumers, end-to-end demo
2. Processors & Composition	25	4+ strategies with proper PCM, chain, builder
3. Testing	25	Unit + integration tests, meaningful coverage
4. UI & Visualization	30	Configuration UI + real-time waveform/spectrum
Total	**100**	

Deductions may apply for:

- Treating individual bytes as audio samples (ignoring 16-bit PCM format)
- Code that compiles but produces noise/silence instead of correct audio
- Poor code organization, missing documentation, or copy-pasted code without understanding
- Tests that don't actually verify audio correctness (e.g., only checking array length)

Bonus points (up to +15) for:

- Exceptionally clean architecture or creative design
- Implementing a genuinely interesting DSP algorithm (reverb, pitch shift, etc.)
- Visual pipeline editor or other innovative UI
- Particularly thorough or creative testing

8 Blueprint Structure

The starter code provides the following (your task is to extend and complete it):

File	Purpose
`audiostream/AudioStream.java`	Core I/O interface
`channel/Channel.java`	Thread-safe data buffer with subchannels
`channel/ObservableChannel.java`	Observer-pattern channel
`processing/AudioProcessingStrategy.java`	Strategy interface
`processing/VolumeControlStrategy.java`	Reference: proper PCM processing
`processing/AudioProcessingContext.java`	Strategy context
`processing/ProcessorChain.java`	*Skeleton*: composite processor
`pipeline/Pipeline.java`	Pipeline interface
`pipeline/AudioPipeline.java`	*Skeleton*: builder-based pipeline
`pin/DataPin.java`	Observer interface
`pin/AudioProcessorPin.java`	Concrete observer
`source/WAVFileReader.java`	Reference source
`source/SineWaveGenerator.java`	Test tone generator
`consumer/FileWriter.java`	Reference consumer
`converter/Converter.java`	Generic converter interface
`converter/SpeechToTextConverter.java`	Stub converter
`config/AudioSystemConfig.java`	Spring configuration
`config/ConfigManagerUI.java`	JavaFX UI skeleton
`util/SystemLogger.java`	Logging utility
`util/WAVUtils.java`	WAV header read/write utilities
`test/.../ChannelTest.java`	Sample: channel unit test
`test/.../VolumeControlStrategyTest.java`	Sample: processor unit test

9 Tips and Resources

- **Start with SineWaveGenerator**: test your processors on generated tones before attempting file I/O or microphone capture. You can verify correctness mathematically (e.g., doubling volume should double sample values).
- **Java Sound API**: `javax.sound.sampled` provides `AudioInputStream`, `AudioSystem`, `SourceDataLine` (playback), and `TargetDataLine` (recording).
- **WAV file format**: A WAV file = 44-byte RIFF header + raw PCM data. Use the provided `WAVUtils` for header manipulation.
- **Threading**: Audio sources and sinks typically run in their own threads. Use `BlockingQueue` (already in Channel) for synchronization. Use `ExecutorService` for managed thread pools.
- **JavaFX Canvas for visualization**: Draw waveforms by mapping sample values to pixel coordinates. Use `AnimationTimer` or `Timeline` for smooth updates.

- **Echo/Delay algorithm hint**: maintain a circular buffer of past samples. Output = current sample + gain $\times$ sample from N positions ago.
- **Useful test pattern**: generate a known sine wave, process it, verify mathematical properties of the output (amplitude, zero-crossings, etc.).